A Practical Introduction to Machine Learning in Python
Day 1 - Monday afternoon
»From text to features«

Rupert Kiddle
Sjoerd Stolwijk

r.t.kiddle@vu.nl, @rptkiddle
s.b.stolwijk@uu.nl

September 15, 2025

Gesis

Today

From text to features: vectorizers

Machine *Learning*: How to teach it?

General idea

Problems

Fix 1: use weighting for informative vs less informative words

Fix 2: Smart tokenization

– Dealing with massive data intermezzo –

Fix 3: Preprocessing to deal with f.e. Capitalization, punctuation

Fix 4: Pruning of less informative words

From text to features: vectorizers

From text to features: vectorizers

Machine *Learning*: How to teach it?

Teaching a toddler

- point it out
- explain in other words
- from the context: You shall know a word by the company it keeps (Firth)
- we need something even simpler

Teaching a toddler

- point it out
- explain in other words
- from the context: You shall know a word by the company it keeps (Firth)
- we need something even simpler

Teaching a toddler

- point it out
- explain in other words
- from the context: You shall know a word by the company it keeps (Firth)
- we need something even simpler

Teaching a toddler

- point it out
- explain in other words
- from the context: You shall know a word by the company it keeps (Firth)
- we need something even simpler

How Humans Learn vs. How Machines Learn

Human Learning	Machine Learning
Example counting (seeing many repetitions)	Rule-based systems / Traditional ML
Pointing out (explicit correction / feedback)	Supervised learning (labels, corrections)
Explaining in other words (paraphrase / abstraction)	Representation learning / Deep Learning
Interpreting context (discourse / world knowledge)	Transformers / Generative AI

How Humans Learn vs. How Machines Learn

Human Learning	Machine Learning
Example counting (seeing many repetitions)	Rule-based systems / Traditional ML
Pointing out (explicit correction / feedback)	Supervised learning (labels, corrections)
Explaining in other words (paraphrase / abstraction)	Representation learning / Deep Learning
Interpreting context (discourse / world knowledge)	Transformers / Generative AI

How Humans Learn vs. How Machines Learn

Human Learning	Machine Learning
Example counting (seeing many repetitions)	Rule-based systems / Traditional ML
Pointing out (explicit correction / feedback)	Supervised learning (labels, corrections)
Explaining in other words (paraphrase / abstraction)	Representation learning / Deep Learning
Interpreting context (discourse / world knowledge)	Transformers / Generative AI

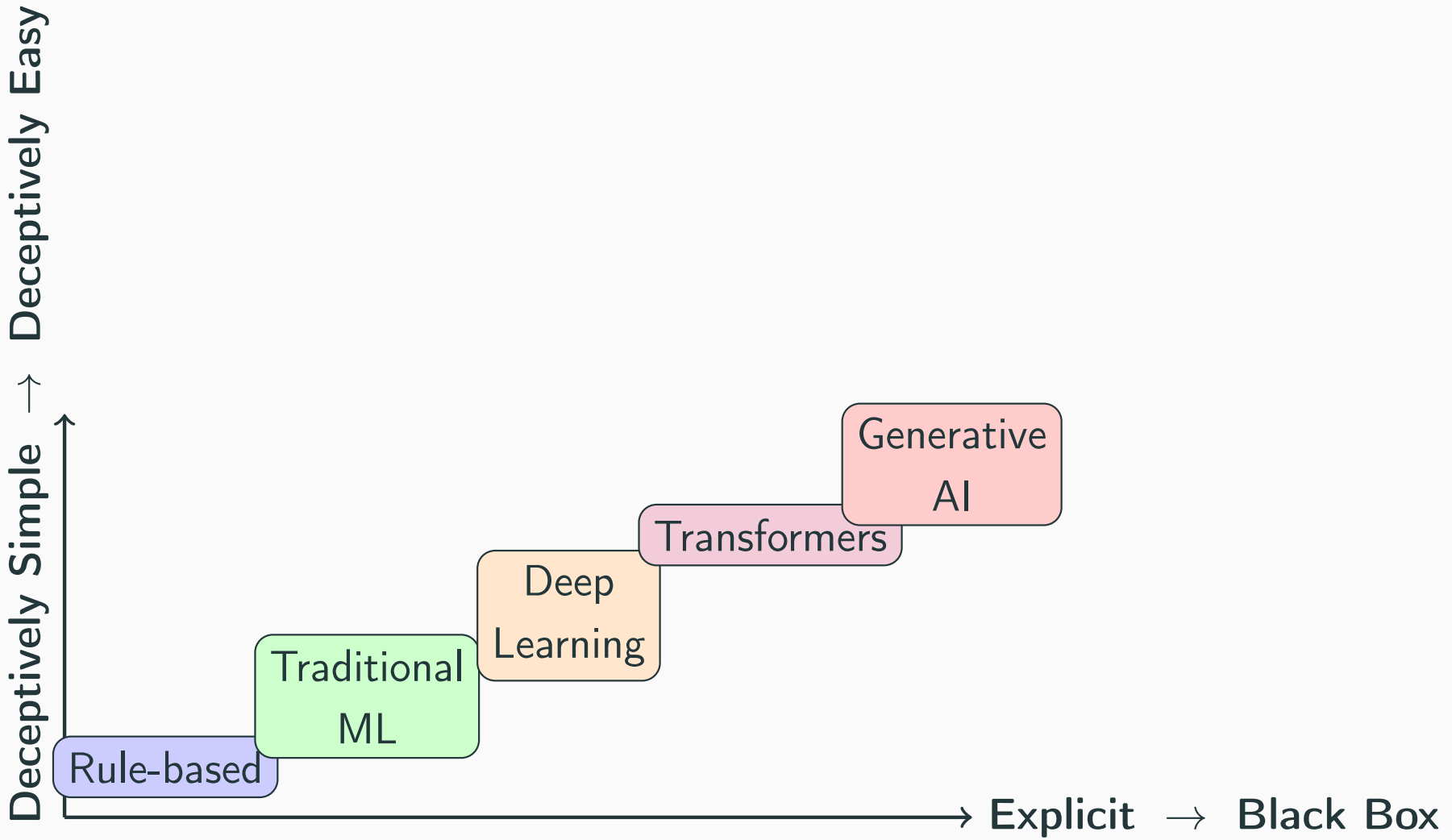
How Humans Learn vs. How Machines Learn

Human Learning	Machine Learning
Example counting (seeing many repetitions)	Rule-based systems / Traditional ML
Pointing out (explicit correction / feedback)	Supervised learning (labels, corrections)
Explaining in other words (paraphrase / abstraction)	Representation learning / Deep Learning
Interpreting context (discourse / world knowledge)	Transformers / Generative AI

How Humans Learn vs. How Machines Learn

Human Learning	Machine Learning
Example counting (seeing many repetitions)	Rule-based systems / Traditional ML
Pointing out (explicit correction / feedback)	Supervised learning (labels, corrections)
Explaining in other words (paraphrase / abstraction)	Representation learning / Deep Learning
Interpreting context (discourse / world knowledge)	Transformers / Generative AI

Evolution of Machine Learning



From text to features: vectorizers

General idea

Bag Of Words (BOW): A text as a collections of words

Let us represent a string

```
t = "This this is is is a test test test"
```

like this:

```
from collections import Counter
print(Counter(t.split()))
```

```
Counter({'is': 3, 'test': 3, 'This': 1, 'this': 1, 'a': 1})
```

Compared to the original string, this representation

- is less repetitive
- preserves word frequencies
- but does *not* preserve word order
- can be interpreted as a vector to calculate with (!!!)

Of course, still a lot of stuff to fine-tune... (for example, This/this)

Bag Of Words (BOW): A text as a collections of words

Let us represent a string

```
t = "This this is is is a test test test"
```

like this:

```
from collections import Counter
print(Counter(t.split()))
```

```
Counter({'is': 3, 'test': 3, 'This': 1, 'this': 1, 'a': 1})
```

Compared to the original string, this representation

- is less repetitive
- preserves word frequencies
- but does *not* preserve word order
- can be interpreted as a vector to calculate with (!!!)

Of course, still a lot of stuff to fine-tune... (for example, This/this)

From vector to matrix: Document Term Matrix (DTM)

If we do this for multiple texts, we can arrange the vectors in a table.

t1 = "This this is is is a test test test"

t2 = "This is an example"

	a	an	example	is	this	This	test
t1	1	0	0	3	1	1	3
t2	0	1	1	1	0	1	0

From vector to matrix: Document Term Matrix (DTM)

What if the text is cyphered?

t1 = "Siht si na elpmaxe"

	a	na	elpmaxe	si	this	Siht	test
t2	0	1	1	1	0	1	0

NOTE: The machine does not understand anything about the meaning of each word

From vector to matrix: Document Term Matrix (DTM)

What if the text is cyphered?

t1 = "Siht si na elpmaxe"

	a	na	elpmaxe	si	this	Siht	test
t2	0	1	1	1	0	1	0

NOTE: The machine does not understand anything about the meaning of each word

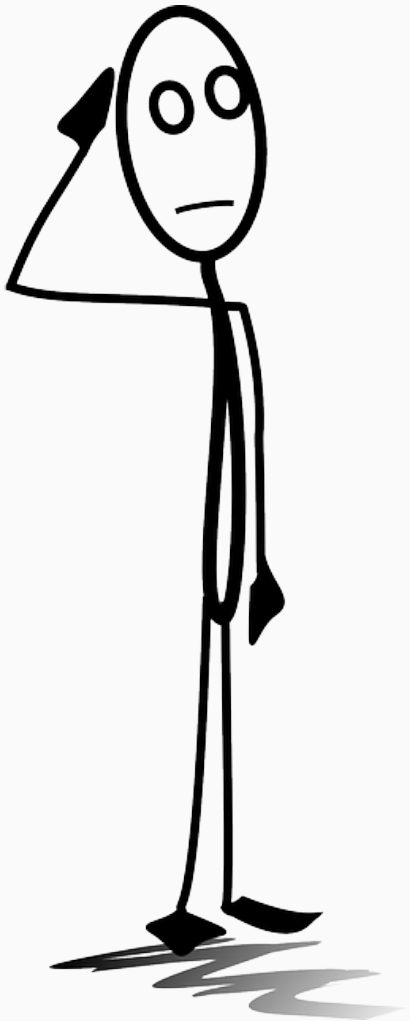
From vector to matrix: Document Term Matrix (DTM)

What if the text is cyphered?

t1 = "Siht si na elpmaxe"

	a	na	elpmaxe	si	this	Siht	test
t2	0	1	1	1	0	1	0

NOTE: The machine does not understand anything about the meaning of each word



What can you do with such a matrix?
Why would you want to represent a collection of texts in such a way?

What is a vectorizer

- Transforms a list of texts into a sparse (!) matrix (of word frequencies)
- Vectorizer needs to be "fitted" to the training data (learn which words (features) exist in the dataset and assign them to columns in the matrix)
- Vectorizer can then be re-used to transform other datasets

What is a vectorizer

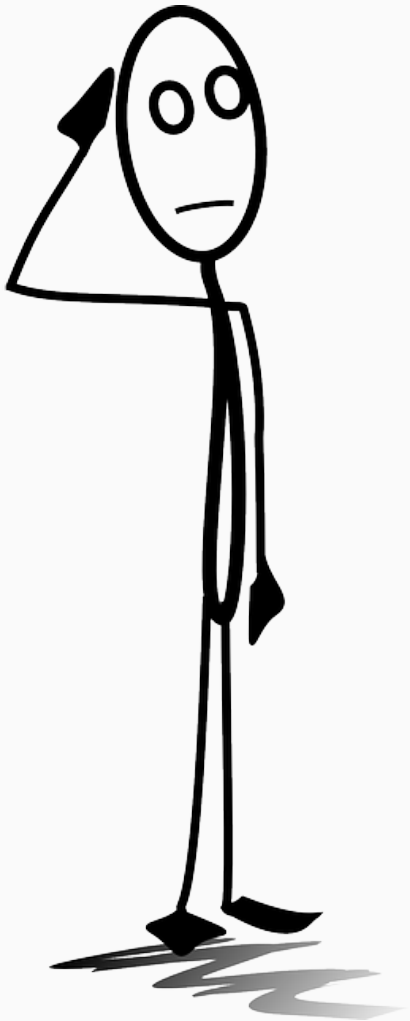
- Transforms a list of texts into a sparse (!) matrix (of word frequencies)
- Vectorizer needs to be "fitted" to the training data (learn which words (features) exist in the dataset and assign them to columns in the matrix)
- Vectorizer can then be re-used to transform other datasets

What is a vectorizer

- Transforms a list of texts into a sparse (!) matrix (of word frequencies)
- Vectorizer needs to be "fitted" to the training data (learn which words (features) exist in the dataset and assign them to columns in the matrix)
- Vectorizer can then be re-used to transform other datasets

The cell entries: raw counts versus tf·idf scores

- In the example, we entered simple counts (the "term frequency")



But are all terms equally important?

From text to features: vectorizers

Problems

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs "biking"
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs "biking"
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs "biking"
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs "biking"
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs "biking"
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs "biking"
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs "biking"
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs "biking"
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs "biking"
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs "biking"
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

Maybe we can do better?

What about:

- negations: “not left” or “didn’t”
- context: “bank” near “river” or near “money”
- synonyms: “bike” or “bicycle”
- spelling mistakes: “never” or “nevr”
- punctuation: “did.” or “did”
- abbreviations: “US” vs “United States” vs “U.S.”
- verb variations: “biked” vs “bikes” vs "biking"
- word derivations: “beauty” versus “beautiful”
- non-informative words like “a”, “and” or “with”
- Capitalization: “Tom, go please” vs “Please go, Tom”
- formatting symbols: “\n” (newline) or “
” (HTML code)

From text to features: vectorizers

Fix 1: use weighting for informative vs less informative words

The cell entries: raw counts versus tf·idf scores

- In the example, we entered simple counts (the "term frequency")
- But does a word that occurs in almost all documents contain much information?
- And isn't the presence of a word that occurs in very few documents a pretty strong hint?
- Solution: Weigh by *the number of documents in which the term occurs at least once) (the "document frequency")*

$\Rightarrow$ we multiply the "term frequency" (tf) by the inverse document frequency (idf)

The cell entries: raw counts versus tf·idf scores

- In the example, we entered simple counts (the "term frequency")
- But does a word that occurs in almost all documents contain much information?
- And isn't the presence of a word that occurs in very few documents a pretty strong hint?
- **Solution: Weigh by *the number of documents in which the term occurs at least once) (the "document frequency")***

$\Rightarrow$ we multiply the "term frequency" (tf) by the inverse document frequency (idf)

The cell entries: raw counts versus tf·idf scores

- In the example, we entered simple counts (the "term frequency")
- But does a word that occurs in almost all documents contain much information?
- And isn't the presence of a word that occurs in very few documents a pretty strong hint?
- **Solution: Weigh by *the number of documents in which the term occurs at least once) (the "document frequency")***

$\Rightarrow$ we multiply the "term frequency" (tf) by the inverse document frequency (idf)

tf·idf

$$w_{i,j} = tf_{i,j} \times \log\left(\frac{N}{df_i}\right)$$

$tf_{i,j} =$ number of occurrences of term i in document j

$df_i =$ number of documents containing term i

$N =$ total number of documents

Is tf·idf always better?

It depends.

- In many scenarios, "discounting" too frequent words and "boosting" rare words makes a lot of sense (most frequent words in a text can be highly un-informative)
- Beauty of raw tf counts, though: interpretability + describes document in itself, not in relation to other documents
- Ultimately, it's an empirical question which works better ($\rightarrow$ machine learning)

Different vectorizers

1. CountVectorizer (=simple word counts)
2. TfidfVectorizer (word counts ("term frequency") weighted by number of documents in which the word occurs at all ("inverse document frequency"))

Different vectorizers

1. CountVectorizer (=simple word counts)
2. TfidfVectorizer (word counts ("term frequency") weighted by number of documents in which the word occurs at all ("inverse document frequency"))

From text to features: vectorizers

Fix 2: Smart tokenization

Vectorizers take care of...

1. Tokenization: Splitting sentences into tokens (terms, words)
2. Vocabulary: Create a list of unique tokens in the corpus
3. Vectorization: For each document, assigned term frequencies or tf·idf scores

Vectorizers take care of...

1. Tokenization: Splitting sentences into tokens (terms, words)
2. Vocabulary: Create a list of unique tokens in the corpus
3. Vectorization: For each document, assigned term frequencies or tf·idf scores

Vectorizers take care of...

1. Tokenization: Splitting sentences into tokens (terms, words)
2. Vocabulary: Create a list of unique tokens in the corpus
3. Vectorization: For each document, assigned term frequencies or tf·idf scores

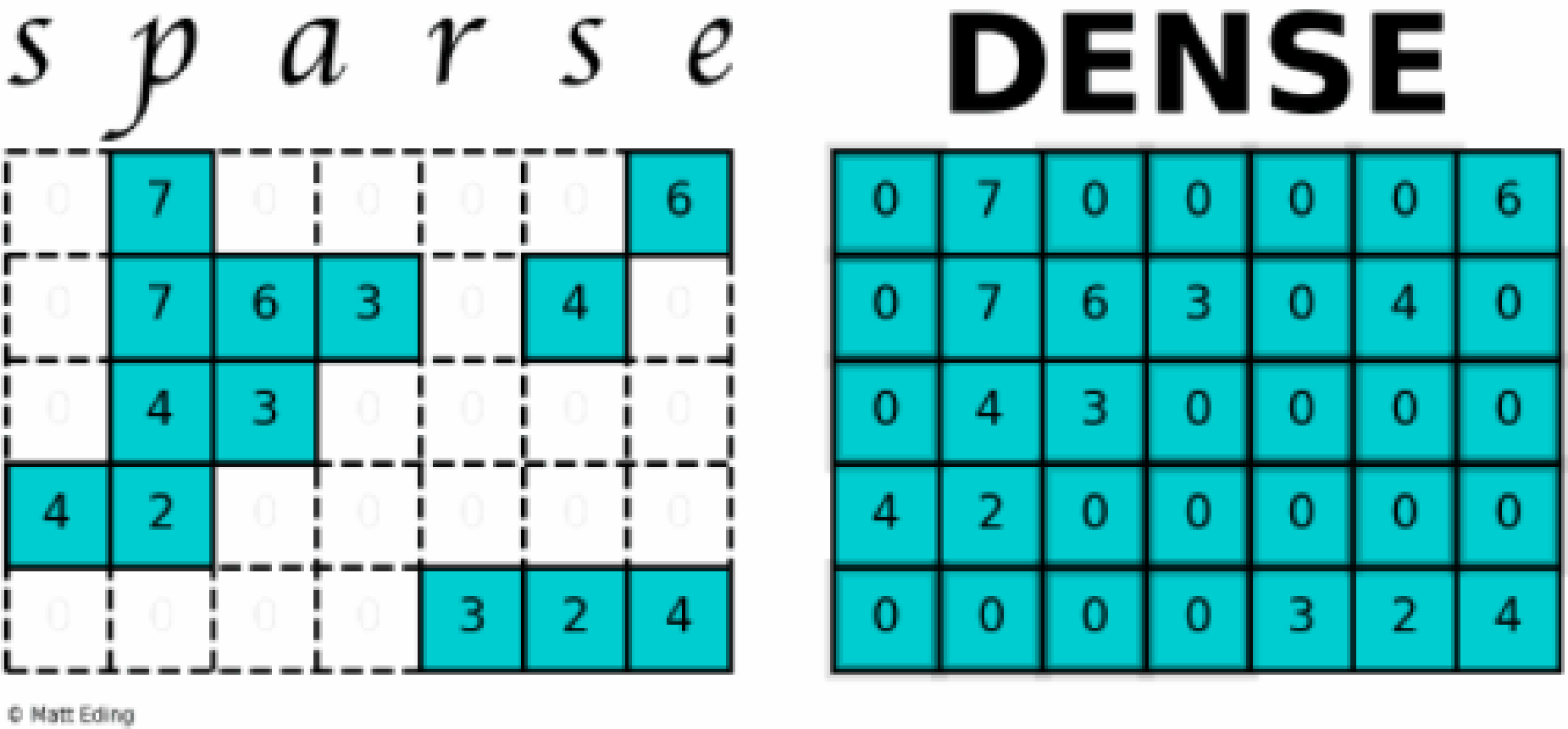
From text to features: vectorizers

– Dealing with massive data intermezzo –

Internal representations

Sparse vs dense matrices

- $\rightarrow$ tens of thousands of columns (terms), and one row per document
- Filling all cells is inefficient *and* can make the matrix too large to fit in memory (!!!)
- Solution: store only non-zero values with their coordinates! (sparse matrix)
- dense matrix (or dataframes) not advisable, only for toy examples



https://matteding.github.io/2019/04/25/sparse-matrices/

From text to features: vectorizers

Fix 3: Preprocessing to deal with f.e. Capitalization, punctuation

OK, good enough, perfect?

scikit-learn's CountVectorizer (default settings)

- applies lowercasing
- deals with punctuation etc. itself
- minimum word length > 1
- more technically, tokenizes using this regular expression: `r"(?u)\b\w\w+\b"` [1]

```
1 from sklearn.feature_extraction.text import CountVectorizer
2 cv = CountVectorizer()
3 dtm_sparse = cv.fit_transform(docs)
```

[1] ?u = support unicode, \b = word boundary

OK, good enough, perfect?

CountVectorizer supports more

- stopword removal
- custom regular expression
- or even using an external tokenizer
- ngrams instead of unigrams

see
https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

Best of both worlds

Use the Count vectorizer with a NLTK-based external tokenizer! (see book)

OK, good enough, perfect?

CountVectorizer supports more

- stopword removal
- custom regular expression
- or even using an external tokenizer
- ngrams instead of unigrams

see
https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

Best of both worlds

Use the Count vectorizer with a NLTK-based external tokenizer! (see book)

From text to features: vectorizers

Fix 4: Pruning of less informative words

General idea

- Idea behind both stopword removal and tf·idf: too frequent words are uninformative
- (possible) downside stopword removal: a priori list, does not take empirical frequencies in dataset into account
- (possible) downside tf·idf: does not reduce number of features

Pruning: remove all features (tokens) that occur in less than X or more than X of the documents

General idea

- Idea behind both stopword removal and tf·idf: too frequent words are uninformative
- (possible) downside stopword removal: a priori list, does not take empirical frequencies in dataset into account
- (possible) downside tf·idf: does not reduce number of features

Pruning: remove all features (tokens) that occur in less than X or more than X of the documents

General idea

- Idea behind both stopword removal and tf·idf: too frequent words are uninformative
- (possible) downside stopword removal: a priori list, does not take empirical frequencies in dataset into account
- (possible) downside tf·idf: does not reduce number of features

Pruning: remove all features (tokens) that occur in less than X or more than X of the documents

General idea

- Idea behind both stopword removal and tf·idf: too frequent words are uninformative
- (possible) downside stopword removal: a priori list, does not take empirical frequencies in dataset into account
- (possible) downside tf·idf: does not reduce number of features

Pruning: remove all features (tokens) that occur in less than X or more than X of the documents

CountVectorizer, only stopword removal

```
1 from sklearn.feature_extraction.text import CountVectorizer,
      TfidfVectorizer
2 myvectorizer = CountVectorizer(stop_words=mystopwords)
```

CountVectorizer, better tokenization, stopword removal (pay attention that stopword list uses same tokenization!):

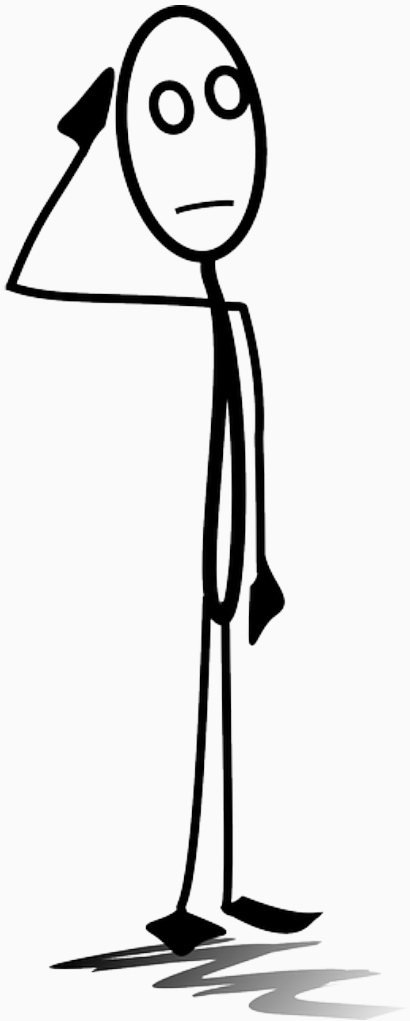
```
1 myvectorizer = CountVectorizer(tokenizer = TreebankWordTokenizer().
      tokenize, stop_words=mystopwords)
```

Additionally remove words that occur in more than 75% or less than $n = 2$ documents:

```
1 myvectorizer = CountVectorizer(tokenizer = TreebankWordTokenizer().
      tokenize, stop_words=mystopwords, max_df=.75, min_df=2)
```

All together: tf·idf, explicit stopword removal, pruning

```
1 myvectorizer = TfidfVectorizer(tokenizer = TreebankWordTokenizer().
      tokenize, stop_words=mystopwords, max_df=.75, min_df=2)
```



What is "best"? Which (combination of) techniques to use, and how to decide?